Python Programming Practical Assignments

All Programs – Proper Format

1. Introduction to Python Programming

1(a) Python Installation – Step-by-Step

```
1. Download Python from the official Python website.
2. Run the Python installer.
3. Select "Add Python to PATH".
4. Click "Install Now".
5. Open Command Prompt.
6. Check installation:

python --version
```

1(b) Program Based on Strings

```
# String operations

name = "Python Programming"

print("String:", name)
print("First character:", name[0])
print("Slicing:", name[0:6])
print("Reverse:", name[::-1])

# String functions
print("Upper:", name.upper())
print("Lower:", name.lower())
print("Length:", len(name))
print("Replace:", name.replace("Python", "Java"))

# Casting
num = "100"
x = int(num)

print("String:", num)
print("Integer:", x)
print("After addition:", x + 50)
```

2. Variables, Operators and Statements

2(a) Variables, Data Types and Operators

```
# Variables
a = 10
b = 5

# Built-in data types
name = "Bhavesh"
marks = 85.5
age = 20
student = True

print("Name:", name)
print("Age:", age)
print("Marks:", marks)
print("Student:", student)

# Arithmetic operators
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Modulus:", a % b)

# Comparison operator
print("a > b:", a > b)
```

```
# Logical operator
print("AND:", a > 5 and b < 10)
```

2(b) Decision and Looping Statements

```
# Decision statement
marks = 75

if marks >= 60:
    print("First Class")
elif marks >= 40:
    print("Pass")
else:
    print("Fail")

# For loop
print("For Loop:")
for i in range(1, 6):
    print(i)

# While loop
print("While Loop:")
i = 1
while i <= 5:
    print(i)
    i += 1
```

3. Function and Organizing Programs

3(a) Tuple, List, Dictionary and Set

```
# List
numbers = [10, 20, 30, 40]
numbers.append(50)
print("List:", numbers)

# Tuple
t = (10, 20, 30)
print("Tuple:", t)

# Dictionary
student = {
    "name": "Bhavesh",
    "age": 20,
    "marks": 85
}
print("Dictionary:", student)

# Set
s = {10, 20, 30, 40}
s.add(50)
print("Set:", s)
```

3(b) Functions with Parameters

```
def add(a, b):
    return a + b

x = 10
y = 20
result = add(x, y)

print("Addition:", result)
```

3(c) Functions Inside Functions

```
def outer():
    print("Outer Function")

    def inner():
        print("Inner Function")

    inner()
```

```
outer()
```

3(d) Built-in Functions

```
numbers = [10, 20, 30, 40, 50]

print("Length:", len(numbers))
print("Maximum:", max(numbers))
print("Minimum:", min(numbers))
print("Sum:", sum(numbers))
print("Sorted:", sorted(numbers))
```

3(e) Anonymous Function – Lambda and Filter

```
numbers = [1, 2, 3, 4, 5, 6]

# Lambda function
square = lambda x: x * x
print("Square of 5:", square(5))

# Filter even numbers
even = list(filter(lambda x: x % 2 == 0, numbers))
print("Even numbers:", even)
```

3(f) Map, List Comprehension and Dictionary

```
numbers = [1, 2, 3, 4, 5]

# Map
square = list(map(lambda x: x * x, numbers))
print("Squares:", square)

# List comprehension
even = [x for x in numbers if x % 2 == 0]
print("Even numbers:", even)

# Dictionary
student = {x: x * x for x in numbers}
print("Dictionary:", student)
```

3(g) Class and Object

```
class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

# Object
s1 = Student("Bhavesh", 85)
s1.display()
```

3(h) Array in Python

```
from array import array

numbers = array('i', [10, 20, 30, 40, 50])

print("Array:", numbers)

numbers.append(60)

print("After adding:", numbers)
print("First element:", numbers[0])
```

3(i) Create, Import and Use Module

```
# File: mymodule.py

def add(a, b):
    return a + b

def message():
```

```
    print("Hello from module")

# Main program: main.py
import mymodule

print("Addition:", mymodule.add(10, 20))
mymodule.message()
```

3(i) Package Example

```
Folder structure:
mypackage/
    __init__.py
    calculator.py
main.py

# calculator.py
def multiply(a, b):
    return a * b

# main.py
from mypackage.calculator import multiply

print("Multiplication:", multiply(5, 4))
```

4. File and Directories

4(a) Writing, Appending and Reading Text Files

```
# Writing to file
file = open("sample.txt", "w")
file.write("Hello Python\n")
file.write("This is a text file.")
file.close()

# Appending to file
file = open("sample.txt", "a")
file.write("\nThis line is appended.")
file.close()

# Reading file
file = open("sample.txt", "r")
data = file.read()
print(data)
file.close()
```

4(b) Paths, Directories and File Operations

```
import os
import shutil

# Create directory
os.mkdir("TestFolder")

# Check directory
print("Exists:", os.path.exists("TestFolder"))

# File information
print("Current Path:", os.getcwd())

# Create file
file = open("TestFolder/test.txt", "w")
file.write("Hello Python")
file.close()

# Rename file
os.rename("TestFolder/test.txt", "TestFolder/new.txt")

# Copy file
shutil.copy("TestFolder/new.txt", "TestFolder/copy.txt")

# Move file
shutil.move("TestFolder/copy.txt", "copy.txt")
```

```
# Remove file
os.remove("copy.txt")

print("File operations completed.")
```

5. Python GUI Programming

5(a) GUI Widgets, Resizing and Configuring Options

```
import tkinter as tk

root = tk.Tk()
root.title("GUI Widgets")
root.geometry("400x300")

label = tk.Label(root, text="Enter Your Name")
label.pack()

entry = tk.Entry(root)
entry.pack()

button = tk.Button(
    root,
    text="Submit",
    command=lambda: label.config(text="Hello " + entry.get())
)
button.pack()

root.mainloop()
```

5(b) Layouts, Packing Order and Widget Appearance

```
import tkinter as tk

root = tk.Tk()
root.title("Layout Example")
root.geometry("400x300")

label1 = tk.Label(root, text="Name")
label1.grid(row=0, column=0)

entry = tk.Entry(root)
entry.grid(row=0, column=1)

button = tk.Button(root, text="Submit")
button.grid(row=1, column=1)

root.mainloop()
```

6. Text Processing and XML

6(a) Text Processing and Searching for Files

```
import os

text = "Python is easy and Python is powerful."

print("Original:", text)
print("Upper:", text.upper())
print("Lower:", text.lower())
print("Python count:", text.count("Python"))

# Search files
for file in os.listdir("."):
    if file.endswith(".txt"):
        print("Text file found:", file)
```

6(b) Processing XML Document Using XML Library

```
import xml.etree.ElementTree as ET

# Create XML
```

```
root = ET.Element("student")

name = ET.SubElement(root, "name")
name.text = "Bhavesh"

marks = ET.SubElement(root, "marks")
marks.text = "85"

tree = ET.ElementTree(root)
tree.write("student.xml")

# Read XML
tree = ET.parse("student.xml")
root = tree.getroot()

print("Name:", root.find("name").text)
print("Marks:", root.find("marks").text)
```

6(c) HTML Parsing

```
from html.parser import HTMLParser

class MyParser(HTMLParser):

    def handle_starttag(self, tag, attrs):
        print("Start tag:", tag)

    def handle_endtag(self, tag):
        print("End tag:", tag)

    def handle_data(self, data):
        if data.strip():
            print("Data:", data)

html = """
<html>
<body>
<h1>Python</h1>
<p>HTML Parsing Example</p>
</body>
</html>
"""

parser = MyParser()
parser.feed(html)
```

7. Accessing Databases

7(a) DBM – Persistent Dictionary

```
import dbm

# Create database
db = dbm.open("studentdb", "c")

# Store data
db["name"] = "Bhavesh"
db["marks"] = "85"
db["course"] = "MCA"

# Access data
print("Name:", db["name"].decode())
print("Marks:", db["marks"].decode())
print("Course:", db["course"].decode())

# Close database
db.close()
```

7(b) Relational Database – SQLite, SQL and Transactions

```
import sqlite3

# Connect to database
conn = sqlite3.connect("student.db")
```

```
# Create cursor
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS student (
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks INTEGER
)
""")

# Insert data
cursor.execute(
    "INSERT INTO student (name, marks) VALUES (?, ?)",
    ("Bhavesh", 85)
)

# Commit transaction
conn.commit()

# Read data
cursor.execute("SELECT * FROM student")
rows = cursor.fetchall()

for row in rows:
    print(row)

# Close connection
conn.close()
```

7(b) Important SQL Statements

```
CREATE TABLE student (...);
INSERT INTO student VALUES (...);
SELECT * FROM student;
UPDATE student SET marks = 90 WHERE id = 1;
DELETE FROM student WHERE id = 1;
```